Exp No: 12 a
Date : 24/9/26

A: Echo client-Server using TCP Sockets

Aim:

To write and implement an Echo client-Server program using Tcp Sockets in C

Result:

Thus, The Tcp and UDP echo client - Server program were successfully implemented and executed in c

Code Implementation (TCP)

tcp_server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define BUFFER_SIZE 1024

int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    char buffer[BUFFER_SIZE] = {0};

    // 1. Create socket
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
    }

    printf("[TCP Server] Waiting for incoming connections on port
%d...\n", PORT);

    // 4. Accept connection
```

Exp: 12b

Date: 24/9/26

Echo client-server using UDP Sockets

Aim:

To write and implement an echo client-server program using UDP Socket in C

Result:

Thus, the UDP echo client-server programs were successfully implemented and executed in C

}

Input/Output (TCP)

Input (Client Terminal):

Enter message to send to server: Hello Networks Lab!

Output (Server Terminal):

[TCP Server] Waiting for incoming connections on port 8080...

[TCP Server] Received from client: Hello Networks Lab!

[TCP Server] Echoed message back to client.

Output (Client Terminal):

[TCP Client] Message sent.

[TCP Client] Echo received from server: Hello Networks Lab!

Part B: Echo Client-Server using UDP Sockets

AIM

To write and implement an Echo Client-Server program using UDP sockets in C.

Algorithm

Server Side

1. Create a UDP socket using socket(AF_INET, SOCK_DGRAM, 0).
2. Bind the socket to an IP address and port number using bind().
3. Receive a message and store the client's address using recvfrom().
4. Echo the message back to the client using sendto().